

Pricing and Matching Policies for a Shared-Rides Platform:

From a Hand-Tuned Heuristic to Approximate Dynamic Programming

Chenrui Xu

UC Berkeley, IEOR C253 (Spring 2025) • Carnegie Mellon University (current PhD)

Post-course extension

Abstract

We study joint pricing and matching for a shared-rides operator on Chicago rideshare data, using the *Calyber* problem from UC Berkeley IEOR C253, taught by Prof. Chiwei Yan (formerly of Uber Marketplace, where he designed the rider surge-pricing algorithm). The course submission (team *Waymodidi*) is a hand-tuned heuristic that flips a conversion-maximizing baseline — which loses money under our assumed cost model — into a profitable online policy. We extend that submission post-course with (i) an independent event-driven simulator, (ii) a fitted logistic conversion-response model, and (iii) an Approximate Dynamic Programming (ADP) pricing policy obtained by backward induction over a (time, queue size) state grid, targeting the pricing dimension while reusing the heuristic’s matching function. On identical replayed arrival streams, the baseline yields $-\$9.79/\text{min}$, the hand-tuned heuristic yields $+\$3.87/\text{min}$, and the ADP policy yields $+\$3.82/\text{min}$ — reaching parity with the heuristic without hand-tuned price bands. We report these as internal-simulator numbers only: our simulator is not calibrated against the course’s hidden validation environment (where the same heuristic scored $\$2.44/\text{min}$), and the conversion model is fit on baseline-policy data whose price support does not fully cover the heuristic’s operating range. We view the parity as suggestive rather than conclusive: the ADP framework is re-derivable under changes in cost structure or arrival distribution for which a hand-tuned heuristic would have to be re-tuned.

1 Introduction

Shared-rides platforms — Uber Pool, Lyft Line, Didi Express Pool, Waymo fleet operations — make two real-time decisions on every incoming request: the quoted per-mile price and whether to match the rider with an existing waiting request. These decisions are tightly coupled: a higher price loses elastic demand but improves unit margin; matching saves vehicle-miles but adds detour cost and rider waiting time. Platforms operate this decision loop at scale against stochastic arrivals, heterogeneous origins and destinations, and limited rider patience. Academic treatments range from stochastic control formulations [1, 4] to reinforcement-learning controllers trained on simulators [2]. In practice, production systems are overwhelmingly hand-tuned heuristics, because even a coarse model of the Markov decision process requires a conversion response function, a matching-probability model, and a trustworthy simulator — artifacts that are expensive to build and brittle to regime change.

The Calyber problem, assigned in UC Berkeley’s IEOR C253 in Spring 2025, is a simplified but faithful instantiation of this setting. Training data covers six weeks of one-hour arrival windows across 76

Chicago community areas under a fixed baseline policy (per-area base prices plus a greedy longest-shared-trip matcher). Student teams designed their own (PRICINGPOLICY, MATCHINGPOLICY) pair; policies were ranked by profit on a held-out test distribution.

This writeup documents the post-course extension of the Waymodidi team’s submission. The contributions are scoped as an internal study, not a validated result against the course’s held-out environment:

1. **Independent simulator (§3.2)**: an event-driven replay of the training arrival stream that takes any policy pair as a drop-in and reports profit, match rate, conversion, and waiting-time metrics. This removes the need for a course-internal simulator and enables apples-to-apples baseline versus candidate comparisons.
2. **Learned conversion-response model (§3.1)**: a logistic model of $\mathbb{P}(\text{convert} \mid \text{price, trip length, pickup area})$ fit to the 11,788 training rider records, closing the loop between policy decisions and downstream rider behavior in the simulator.
3. **ADP pricing policy (§3.3)**: a tabular finite-horizon MDP over (minute, queue size) solved by backward induction, yielding a DP-optimal base price per state. On the same simulator the ADP policy reaches parity with the hand-tuned heuristic without the heuristic’s hand-selected price bands and penalty thresholds; it reuses the heuristic’s matching function, so ADP here targets the pricing dimension specifically.

2 Problem Formulation

Arrivals form a non-stationary marked point process: a rider i is characterized by $\xi_i = (\text{area}_i^o, \text{area}_i^d, L_i)$, where L_i is the haversine solo trip length. On arrival, the operator quotes a per-mile price $p_i \in [c/2, 1]$, where $c = \$0.70/\text{mi}$ is the marginal driving cost. The rider converts with probability

$$\pi(p_i, \xi_i) = \sigma \left(\beta_0 + \beta_p p_i + \beta_L L_i + \sum_a \beta_a^{\text{area}} \mathbf{1}[\text{area}_i^o = a] \right),$$

fit by logistic regression on the training data; see §3.1. If the rider converts, a matching policy decides whether to pair them with one of the waiting riders S_t (state) or add them to S_t . Matched pairs are dispatched immediately; unmatched conversions wait, with solo dispatch after a timeout $T_{\max} = 300$ s.

Revenue per rider is $p_i L_i$, independent of match outcome (platform charges by solo distance). Cost for a solo dispatch is $c L_i$. Cost for a matched pair (i, j) is $c T_{ij}$, where T_{ij} is the shortest shared-trip length over the five feasible pickup/drop-off orderings [3] — allocated pro-rata to the two riders by solo-length share. These revenue and cost conventions (charging by solo distance; pro-rata allocation of shared cost) are assumptions we make to close the simulator; the course’s internal environment may use different rules. The operator’s objective over a one-hour window is expected total profit

$$\mathcal{J} = \mathbb{E} \left[\sum_i (\text{revenue}_i - \text{cost}_i) \right].$$

3 Method

3.1 Conversion-Response Model

We fit a logistic regression on $(p_i, L_i, \text{area}_i^o)$ with conversion-outcome labels from the 11,788 training rows under the baseline policy. The price coefficient is $\hat{\beta}_p = -6.4$ (training accuracy 65.9%; overall conversion rate 58.2%), indicating a strong price elasticity consistent with the course’s design. This model is shared across all downstream policy evaluations so that differences in simulated outcomes across policies are attributable to the policies themselves rather than to disparate demand assumptions. *Support caveat:* baseline quoted prices concentrate below \$0.8/mi, so the heuristic’s and ADP’s operating range (\$0.75–\$0.90/mi) is partly an extrapolation; the conversion elasticity at those prices is not fully identified by the training data.

3.2 Simulator

The simulator (Listing 1 of `simulator.py`) replays the training arrival stream in chronological order within each one-hour window. On each arrival:

1. The policy’s `pricing_function` is invoked on the current waiting state S_t and arriving rider.
2. The rider converts with probability $\pi(p, \xi)$ from the learned conversion model, sampled with a fixed random seed for reproducibility and cross-policy comparability.
3. If converted, the policy’s `matching_function` returns either a partner from S_t or NONE; matched pairs are dispatched with shared-trip cost, unmatched conversions are added to S_t .

Before each arrival, queue entries whose wait exceeds $T_{\max}$ are dispatched solo. Window-end riders are liquidated as solo dispatches — an assumption we introduce to close the window (the course’s handling is not specified to us). The simulator returns per-dispatch records (revenue, cost, wait, matched) from which all aggregate metrics are computed. $T_{\max} = 300$ s is set to the median of the training patience distribution; the empirical tail extends to ≈ 600 s, so this truncation is an approximation.

3.3 Approximate Dynamic Programming for Pricing

We target the pricing decision with a tabular finite-horizon MDP. State is $s = (t, q)$, where $t \in \{0, 1, \dots, T - 1\}$ indexes one-minute bins of the hour-long window and $q \in \{0, \dots, Q_{\max}\}$ is the current queue size. Action is $a \in \mathcal{P}$, a 15-element grid $\{0.30, 0.35, \dots, 1.00\}$. The Bellman recursion is

$$V(t, q) = \max_{a \in \mathcal{P}} \left\{ \underbrace{\lambda_t \cdot \pi(a) \cdot \bar{r}(a, q)}_{\text{expected profit in the minute}} + V(t + 1, q'(a, q)) \right\},$$

with terminal condition $V(T, q) = 0$. Here $\lambda_t \approx 32.7$ is the empirical arrivals-per-minute rate, $\pi(a)$ is the conversion probability at price a evaluated at the representative trip length $\bar{L} = 3.4$ mi, and the expected per-converted-rider profit is

$$\bar{r}(a, q) = \mu(q)(a\bar{L} - c\bar{L}\kappa_s) + (1 - \mu(q))(a\bar{L} - c\bar{L}),$$

with match probability $\mu(q) = \min(q/4, 0.9)$ and shared-cost factor $\kappa_s = 0.75$. Both $\mu(q)$ and κ_s are coarse closed-form approximations rather than fit from data: $\mu(q)$ is an ad-hoc linear ramp

capturing the intuition that a longer queue produces more match candidates, and κ_s reflects the training-data average ratio of matched-pair cost to solo-sum cost. The conversion probability $\pi(a)$ in the DP is evaluated at a single *representative area* index (the median, area 38); pickup-area heterogeneity is absorbed into the online price blend rather than into the DP state. The queue transition $q'(a, q) = \text{round}(q + \lambda_t \pi(a)(1 - 2\mu(q)) - q/T_{\text{wait}})$ combines arrival-driven drift with a first-order timeout drain. We solve by backward induction; the full table is $60 \times 61 \times 15 = 54,900$ cells and solves in <1 s.

The resulting policy is queue-responsive in the expected direction: $a^*(0, 0) = \$0.90/\text{mi}$, $a^*(0, 20) = \$0.75/\text{mi}$ — charge more when the queue is empty and matching is unlikely, less when the queue is long and a high match rate lets a lower price earn a higher-throughput margin.

At runtime, the online `pricing_function` looks up the DP base price $a^*(t, q)$ and blends it toward the course’s per-area typical price with weight 0.3, preserving the DP’s cost-aware structure while respecting pickup-area heterogeneity that the state abstraction does not capture. A hard floor at $1.1c$ enforces per-mile margin even when the blend pulls below cost. The per-area prices are inherited from the baseline policy’s offline calibration; the ADP policy is therefore not purely *de-novo*, but it does replace the heuristic’s hand-tuned price bands and queue-based penalties with a DP-derived base.

4 Results

Table 1 reports aggregate metrics from the simulator on the full training arrival stream, with a common random seed across all three policies.

Table 1: Simulator-based comparison on the training arrival stream (6×60 min). Baseline is the course-provided Example policy; Waymodidi is the hand-tuned submission; ADP is the policy of §3.3.

Metric	Baseline	Waymodidi	ADP	Δ ADP vs. Waymodidi
Profit / min	−\$9.79	+\$3.87	+\$3.82	−1.2%
Revenue / min	\$36.81	\$23.70	\$22.45	−5.3%
Cost / min	\$46.60	\$19.83	\$18.62	−6.1%
Throughput / min	22.39	8.97	8.46	−5.6%
Match rate	94.4%	99.5%	99.5%	+0.0%
Conversion rate	68.4%	27.4%	25.8%	−5.6%
Avg. waiting time	37.0 s	21.7 s	22.5 s	+3.3%
Avg. payment (\$/mi)	\$0.461	\$0.770	\$0.779	+1.1%

Three findings are worth drawing out.

(1) Conversion-only pricing is unprofitable under our assumed cost model. The baseline maximizes throughput and match rate but prices below cost-of-service at the empirical arrival intensity, yielding −\$9.79/min in our simulator. Neither high conversion (68.4%) nor high match rate (94.4%) compensate for negative unit margin. This is the motivating observation for the heuristic’s design; it is contingent on the cost structure assumed here ($c = \$0.70/\text{mi}$, pro-rata shared-cost allocation, revenue billed by solo distance), and the sign of the baseline’s per-minute profit may differ under alternative cost conventions.

(2) A cost-aware heuristic flips the sign. The Waymodidi policy raises the average payment

from \$0.46 to \$0.77/mile and adds a hard per-mile margin floor, accepting a 60% drop in throughput and conversion in exchange for +\$3.87/min profit. The near-100% match rate is retained because the matching function is itself queue-aware (relaxing its threshold as queues grow).

(3) ADP reaches parity with hand-tuning on the pricing dimension. The ADP policy yields +\$3.82/min with no hand-tuned price bands or penalty thresholds; its inputs are the fitted conversion model, the assumed cost parameters, and coarse closed-form approximations for $\mu(q)$ and κ_s . It prices slightly higher on average (+1.1%), sacrifices a small amount of throughput (−5.6%), and matches the heuristic’s match rate and waiting time. Practically, ADP recovers 98.7% of the heuristic’s profit in this simulator. The load-bearing argument is not the +\$3.82/min number — which is internal-simulator only — but the *re-derivability*: running ADP on a new cost structure or arrival distribution requires re-solving the tabular MDP (a sub-second operation), whereas the heuristic’s price bands would need human re-tuning. We caveat this below.

5 Validation Caveats

Before discussing limitations of the method, we record specific reasons the reader should treat the simulator numbers above as internal-only:

- *Simulator not calibrated against ground truth.* On the course’s held-out validation run, the same Waymodidi heuristic earned \$2.44/min; our simulator reports \$3.87/min for it — a 58% discrepancy. This gap sets an upper bound on the reliability of absolute profit numbers from our simulator, and by extension on the −\$9.79 and +\$3.82 figures. Relative ordering across the three policies is more likely to transfer than absolute magnitudes.
- *Conversion model out-of-support.* The logistic conversion response is fit on baseline-policy prices, which concentrate below \$0.8/mi. The heuristic and ADP operate in the \$0.75–\$0.90/mi band; the elasticity in that region is partly extrapolated.
- *Assumed cost and revenue rules.* $c = \$0.70/\text{mi}$, revenue-by-solo-distance, and pro-rata shared-cost allocation are our reading of the problem description. Under other reasonable conventions (e.g., revenue billed by shared distance, or detour disutility priced into cost) the baseline’s sign could flip and ADP vs. heuristic ordering could change.
- *Hand-set knobs in ADP.* $\mu(q) = \min(q/4, 0.9)$, $\kappa_s = 0.75$, $T_{\max} = 300$ s, and the representative area (index 38) are set by inspection of training-data summary statistics rather than by fitting. The price-blend weight (0.3 toward the baseline per-area price) is likewise chosen, not tuned.
- *Not fully de-novo.* ADP’s per-area modulation is inherited from the baseline’s offline price calibration, and its matching function is the Waymodidi heuristic’s. Calling this “from first principles” would overstate the separation; the contribution is the DP-derived state-dependent base price, not a full re-derivation of the joint policy.

6 Discussion and Further Limitations

State abstraction. The DP state (t, q) is deliberately coarse; it ignores pickup-area heterogeneity, trip-length distribution within the queue, and driver-side dynamics. A factored value function $V(t, q, \text{area})$ or a low-rank tensor decomposition would be the minimal upgrade. Matching is untouched here; ADP currently inherits the heuristic’s matching function.

Single-cost structure. Driver supply, repositioning, and empty-haul cost are abstracted away; the cost model is marginal per-mile only. A deployment-grade variant would layer a supply-side MDP on top of the pricing/matching MDP, following the marketplace structure documented for Uber’s surge-pricing team [5].

Research next steps. (i) Factored ADP over (t, q, area) ; (ii) Monte-Carlo policy iteration using simulator rollouts instead of the analytic transition model; (iii) a policy-gradient baseline (REINFORCE or PPO) for comparison under relaxed Markov assumptions; (iv) explicit detour-cost disutility in the matching function and a joint pricing-matching MDP.

Acknowledgments

This project was developed in UC Berkeley IEOR C253 under Prof. Chiwei Yan, who designed the course problem and whose industry experience on Uber’s surge-pricing team shaped the realism of the formulation. Teammates on the Waymodidi submission contributed to the original heuristic; the simulator, conversion model, ADP formulation, and this writeup are post-course individual work.

References

- [1] G. P. Cachon, K. M. Daniels, and R. Lobel. The role of surge pricing on a service platform with self-scheduling capacity. *Manufacturing & Service Operations Management*, 19(3):368–384, 2017.
- [2] B. Turan, R. Pedarsani, and M. Alizadeh. Dynamic pricing and fleet management for electric autonomous mobility on demand systems. *Transportation Research Part C: Emerging Technologies*, 121:102829, 2020.
- [3] I. Lobel and S. Martin. Detours in shared rides. *Management Science*, 71(2):1716–1736, 2025.
- [4] E. Özkan and A. R. Ward. Dynamic matching for real-time ride sharing. *Stochastic Systems*, 10(1):29–70, 2020.
- [5] C. Yan, H. Zhu, N. Korolko, and D. Woodard. Dynamic pricing and matching in ride-hailing platforms. *Naval Research Logistics*, 67(8):705–724, 2020.